PCA of the BANKSY matrix with the azimuthal Gabor filter, one chunk at a time

Randomized SVD with chunk recomputation, explained slowly

Working notes

31 October 2026

The whole idea in one paragraph

The BANKSY matrix with the AGF is far too big to store (about 1.2 TB for 10 million cells and 5,000 genes). The current "lazy" trick avoids storing it, but only works without the AGF, because the AGF contains an absolute value that breaks the algebra the trick relies on. There is a different way to avoid storing it: *any group of cells' columns of the matrix can be rebuilt cheaply from the expression data and the neighbour lists.* So we can sweep through the cells in chunks, rebuild each chunk, use it, and throw it away. A *randomized SVD* needs only a handful of such sweeps (5–7), because it multiplies the matrix by about 60 vectors at once instead of one at a time. The answer reproduces the leading, biologically meaningful principal components accurately. The trailing components, which sit in a flat "noise bulk", are not pinned down exactly; that changes the kNN graph about as much as using 35–40 instead of 50 PCs, and changes clustering a little. If an exact match to the standard pipeline is needed, a second chunked method (the feature covariance) gives it, at about 20 times the arithmetic.

How to read this. Sections 1–2 set up the objects and explain why the AGF is a problem. Section 3 gives the chunk idea, which is simple and exact. Section 4 explains randomized SVD from scratch, assuming only that you know what an SVD is (Appendix A is a refresher if not). Section 5 puts the two together, pass by pass. Sections 6 and 7 cover cost and accuracy, Section 8 checks every claim on a simulated dataset, and Section 9 says what this means for Banksy. Appendix B is a glossary of every technical term.

Corrections to the earlier chat explanation. Writing this up properly, with numbers, changed some things I said before. They are flagged where they come up; in short: (i) the PCA is not "exact": the *features* are exact, but the solver is iterative and the trailing PCs are only approximately recovered (Section 7); (ii) the number of passes is $q + 3$, i.e. 5–7 for 2–4 power iterations; (iii) the working memory is a few GB, the $n \times \ell$ matrix is only the output; (iv) the covariance method costs about 60 times more per pass but about 20 times more in total, not 60.

Contents

1 The setting: what matrix are we taking the PCA of? 3

2 Why the lazy trick does not work with the AGF 5

3 The key observation: any chunk of Z is cheap to rebuild **7**

4 Randomized SVD from scratch **8**

5 The chunked algorithm, pass by pass **10**

6 Costs **13**

7 Accuracy **15**

8 A numerical demonstration **16**

9 What this means for Banksy **21**

A Linear algebra refresher **22**

B Glossary **23**

C The core of the chunked algorithm in code **24**

1 The setting: what matrix are we taking the PCA of?

1.1 Expression data

We have n cells and g genes. The expression matrix is

$$X \in \mathbb{R}^{g \times n}, \qquad X_{ab} = \text{(normalised) expression of gene } a \text{ in cell } b.$$

Each **column is a cell**, each **row is a gene**. This is the Banksy and Seurat convention, and it matters for everything below: when I say "a chunk of columns" I mean "a group of cells". We write $x_b \in \mathbb{R}^g$ for column b, the expression profile of cell b.

X is **sparse**: most entries are zero. In the Xenium Prime 5K benchmarks about 96% of entries are zero, so each cell has roughly $0.04 \times 5000 = 200$ nonzero genes. A sparse matrix stores only the nonzeros (value plus position), so X itself is affordable: about 2×10^9 nonzeros at 10 million cells, roughly 24 GB.

1.2 Spatial neighbours, weights and angles

For each cell j, BANKSY finds its k nearest cells in space, $\mathcal{N}(j)$, and gives each neighbour $i \in \mathcal{N}(j)$ a weight $w_{ij} \geq 0$ with $\sum_{i \in \mathcal{N}(j)} w_{ij} = 1$. With the default kernel (`kNN_median`) the weight decays with distance d_{ij}:

$$w_{ij} \propto \exp\Big(- \frac{d_{ij}^2}{\text{median}_{i' \in \mathcal{N}(j)}\, d_{i'j}^2} \Big), \qquad \text{normalised so the weights of cell } j \text{ sum to 1.}$$

For the AGF we also need the **angle** $\varphi_{ij} \in [0, 2\pi)$ of the arrow from cell j to neighbour i (`getPhi` in `R/computation.R`: `atan2` of the coordinate differences). Banksy uses different k for the two neighbourhood blocks; the usual choice is `k_geom = c(15, 30)`: $k = 15$ for H_0 and $k = 30$ for H_1. Each block has its own neighbour lists and weights.

1.3 The three blocks of features

For every cell j, BANKSY [8] builds three vectors of length g:

1. **Own expression**: x_j.
2. **Neighbour mean** H_0: the weighted average of the neighbours' expression,

$$(H_0)_{\cdot j} = \sum_{i \in \mathcal{N}_0(j)} w_{ij}\, x_i.$$

   In matrix form, put the weights in a sparse $n \times n$ matrix W with $W_{ij} = w_{ij}$ (row i = neighbour, column j = cell, k nonzeros per column). Then $H_0 = XW$. Each column of W sums to one ("column-stochastic").[1]

3. **AGF** H_1 (azimuthal Gabor filter, harmonic $m = 1$): for each gene a,

$$(H_1)_{aj} \;=\; \Big| \sum_{i \in \mathcal{N}_1(j)} w_{ij}\, e^{\mathrm{i}\varphi_{ij}}\, (X_{ai} - \bar{x}_{aj}) \Big|, \qquad \bar{x}_{aj} = \frac{1}{k} \sum_{i \in \mathcal{N}_1(j)} X_{ai}. \tag{1}$$

[1] Banksy's standard path actually computes $|XW|$ (`abs(gcm %*% W)` in `computeHarmonics`). For nonnegative data, such as counts or log-normalised expression, the absolute value does nothing. For data that can be negative, such as z-scored protein data (CODEX, issue #66), the standard path and the lazy path disagree: the lazy path uses XW without the absolute value. Worth raising with Joseph; it is not part of this note's argument.

Here $\mathrm{i} = \sqrt{-1}$, $e^{\mathrm{i}\varphi} = \cos\varphi + \mathrm{i}\sin\varphi$ is a unit arrow pointing at angle φ, and $|\cdot|$ is the length (modulus) of a complex number. Note the centring uses the *unweighted* mean of the neighbours (`center = TRUE`, the default).

What the AGF measures. Think of each neighbour as an arrow pointing from cell j towards that neighbour, scaled by how much more (or less) that neighbour expresses gene a than the local average. Add all the arrows up. If expression is the same all around the cell, the arrows cancel and $H_1 = 0$. If expression is higher on one side, the arrows add up to a long arrow pointing towards the high side, and H_1 is its length. So the AGF measures *how lopsided expression is around the cell*: it is large at boundaries and along gradients, and it does not care which direction the gradient points.

The AGF as one complex sum. Expanding (1):

$$\sum_i w_{ij} e^{\mathrm{i}\varphi_{ij}} (X_{ai} - \bar{x}_{aj}) = \sum_i w_{ij} e^{\mathrm{i}\varphi_{ij}} X_{ai} - \bar{x}_{aj}\, s_j, \qquad s_j := \sum_i w_{ij} e^{\mathrm{i}\varphi_{ij}},$$

and since $\bar{x}_{aj} = \frac{1}{k}\sum_i X_{ai}$, this is $\sum_i c_{ij} X_{ai}$ with

$$c_{ij} \;=\; w_{ij} e^{\mathrm{i}\varphi_{ij}} - \frac{s_j}{k}. \tag{2}$$

So with a sparse complex matrix C ($C_{ij} = c_{ij}$, same sparsity pattern as the $k = 30$ neighbour graph):

$$\boxed{H_1 = |XC| \quad \text{(absolute value taken entry by entry).}} \tag{3}$$

This is exactly what `computeHarmonics` does (its `W_re`, `W_im`, `U`, `s_re`, `s_im`). Two consequences that we use later:

- **Everything before the absolute value is linear in X.** The only nonlinear step is $|\cdot|$.
- **The coefficients of each cell sum to zero**: $\sum_i c_{ij} = s_j - k \cdot s_j/k = 0$. That is the algebraic form of "a gene expressed uniformly around the cell gives $H_1 = 0$". (The demo checks both this and the equivalence of (1) and (3) to $\sim 10^{-16}$.)

1.4 Scaling, weighting, stacking: the BANKSY matrix Z

Each of the three blocks (X, H_0, H_1) is **z-scored row by row**: for every feature (row) f, subtract its mean over all cells and divide by its standard deviation over all cells:

$$\mu_f = \frac{1}{n}\sum_{j=1}^{n} R_{fj}, \qquad \sigma_f = \sqrt{\frac{1}{n-1}\sum_{j=1}^{n}(R_{fj} - \mu_f)^2}, \qquad \tilde{R}_{fj} = \frac{R_{fj} - \mu_f}{\sigma_f},$$

where R is the unscaled block. (Rows with $\sigma_f = 0$ are constant; they become rows of zeros.) The blocks are then multiplied by weights determined by λ (`getLambdas`); with the AGF ($M = 1$) these are

$$a_{\text{own}} = \sqrt{1-\lambda}, \qquad a_{H_0} = \sqrt{2\lambda/3}, \qquad a_{H_1} = \sqrt{\lambda/3}.$$

(At $\lambda = 0.8$: 0.447, 0.730, 0.516.) Finally the three blocks are stacked on top of each other:

$$Z \;=\; \begin{bmatrix} \sqrt{1-\lambda}\,\operatorname{zscore}(X) \\ \sqrt{2\lambda/3}\,\operatorname{zscore}(H_0) \\ \sqrt{\lambda/3}\,\operatorname{zscore}(H_1) \end{bmatrix} \in \mathbb{R}^{F \times n}, \qquad F = 3g. \tag{4}$$

F is the number of **features** (rows). Column j of Z, call it $z_j \in \mathbb{R}^F$, is cell j's full BANKSY profile. It is convenient to write the whole construction as

$$Z = D\,(R - \mu \mathbf{1}^\top), \qquad D = \operatorname{diag}\Big(\frac{a_f}{\sigma_f}\Big), \tag{5}$$

where $R = [X;\, H_0;\, H_1]$ is the unscaled stack, $\mu \in \mathbb{R}^F$ the row means, $\mathbf{1}$ the all-ones vector, and a_f the block weight of row f. "Subtract a vector from every column, then rescale every row" is all z-scoring is.

Remark 1.1 (Clipping). The Seurat wrapper additionally caps z-scores at 10 (upper tail only, matching Seurat's `ScaleData`). That is one more entry-by-entry operation after z-scoring. It is irrelevant to the lazy trick (which cannot handle it either without a correction term), and, as we will see, it costs nothing in the chunked method.

1.5 What "PCA of Z" means

Every real matrix has a **singular value decomposition** (SVD)

$$Z = U\Sigma V^\top = \sum_i \sigma_i\, u_i v_i^\top,$$

with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ (the **singular values**), $u_i \in \mathbb{R}^F$ orthonormal (the **loadings**: directions in feature space) and $v_i \in \mathbb{R}^n$ orthonormal (directions in cell space). Because the rows of Z are already centred, PCA of the cells *is* the truncated SVD: the top p principal components are $u_1, \ldots, u_p$, and each cell's coordinates on them, the **embedding**, are

$$\text{embedding of cell } j \;=\; U_p^\top z_j \in \mathbb{R}^p, \qquad \text{i.e.} \qquad \text{all embeddings} = Z^\top U_p = V_p \Sigma_p \in \mathbb{R}^{n\times p}. \tag{6}$$

(The second form: $Z^\top u_i = \sigma_i v_i$.) This is the matrix stored as `PCA_M1_lam0.8` in Banksy, or as the reduction in Seurat. Clustering and UMAP use only these embeddings.

Key idea

Downstream, everything is computed from $Z^\top U_p$: *each cell's BANKSY profile projected onto the top p loading directions.* So what we must find is U_p, a small $F \times p$ matrix. If we can get it, the embeddings follow from one more look at the data.

1.6 Why we cannot just store Z

Z is **dense**: after centring, even a gene that is zero in a cell gets the value $-\mu_f/\sigma_f \neq 0$. With $n = 10^7$ cells and $g = 5000$ genes, $F = 15{,}000$ and

$$F \times n \times 8 \text{ bytes} = 15{,}000 \times 10^7 \times 8 = 1.2 \times 10^{12} \text{ bytes} = 1.2 \text{ TB}.$$

That is the problem the whole note is about.

2 Why the lazy trick does not work with the AGF

2.1 What an iterative SVD solver needs

Solvers like `irlba` [6] (the one Banksy and Seurat use) never look at the matrix directly. They only ask for two kinds of products, many times over:

$$\text{forward: } Zv \;(v \in \mathbb{R}^n,\ \text{result in } \mathbb{R}^F), \qquad \text{transpose: } Z^\top u \;(u \in \mathbb{R}^F,\ \text{result in } \mathbb{R}^n).$$

If you can compute these products without storing Z, you can run the solver without storing Z.

2.2 The lazy trick, without the AGF

Without the AGF, $Z = [\, a_1 D_1(X - \mu_1 \mathbf{1}^\top);\ a_2 D_2(XW - \mu_2 \mathbf{1}^\top)\,]$. The forward product of the second block is

$$D_2\big(XWv - \mu_2(\mathbf{1}^\top v)\big) = D_2\big(X(Wv) - \mu_2\,(\mathbf{1}^\top v)\big).$$

The point is the brackets: $(XW)v = X(Wv)$. You first compute Wv (cheap: W has k nonzeros per column), then $X(Wv)$ (cheap: X is sparse). You never form XW. The transpose product works the same way: $(XW)^\top u = W^\top(X^\top u)$. That is the whole lazy path (`R/lazy.R`).

2.3 Why the AGF breaks it

The trick needs the block to be "X times some fixed sparse matrix". For the AGF we would need a matrix B with $H_1 = XB$ for every possible X. No such B exists, because the map $X \mapsto |XC|$ is not linear.

Worked example: the AGF is not additive

Take one cell with exactly two neighbours, one directly to the right ($\varphi = 0$) and one directly to the left ($\varphi = \pi$), equal weights $\frac{1}{2}$. Then $s_j = \frac{1}{2}e^0 + \frac{1}{2}e^{\mathrm{i}\pi} = \frac{1}{2} - \frac{1}{2} = 0$, so by (2) the coefficients are just $c_{\text{right}} = \frac{1}{2}$, $c_{\text{left}} = -\frac{1}{2}$.

- Gene expressed (value 1) only in the right neighbour: $H_1 = |\frac{1}{2}| = \frac{1}{2}$.
- Only in the left neighbour: $H_1 = |-\frac{1}{2}| = \frac{1}{2}$.
- In both (the sum of the two situations): $H_1 = |\frac{1}{2} - \frac{1}{2}| = 0$.

A linear map would give $\frac{1}{2} + \frac{1}{2} = 1$ for the sum. The AGF gives 0, because uniform expression around the cell is not lopsided. So no matrix B can reproduce it, and $H_1 v$ cannot be rewritten as $X(\text{something} \cdot v)$.

2.4 Why approximating the absolute value by a series does not rescue it

Write $|z| = \sqrt{t}$ with $t = (\operatorname{Re} z)^2 + (\operatorname{Im} z)^2$. One could try to approximate $\sqrt{t}$ by a polynomial $a_0 + a_1 t + a_2 t^2 + \cdots$. Two things go wrong.

1. **The squared terms already do not factor.** Row a of $(XC) \circ (XC)$ times v (where $\circ$ is entry-by-entry multiplication) equals $x_{a\cdot}\,(C\,\mathrm{diag}(v)C^\top)\,x_{a\cdot}^\top$: a quadratic form around a matrix that changes with v at every iteration and has about $k^2 = 900$ nonzeros per column. Each product becomes hundreds of times more expensive than the lazy one, and higher powers t^2, t^3 are worse.
2. **$\sqrt{t}$ is badly behaved at $t = 0$**, and sparse data put many entries at or near zero. A polynomial matches $\sqrt{t}$ poorly exactly where most of the data sit.

So we drop the requirement that each product be cheap, and look for a method that needs *few* products.

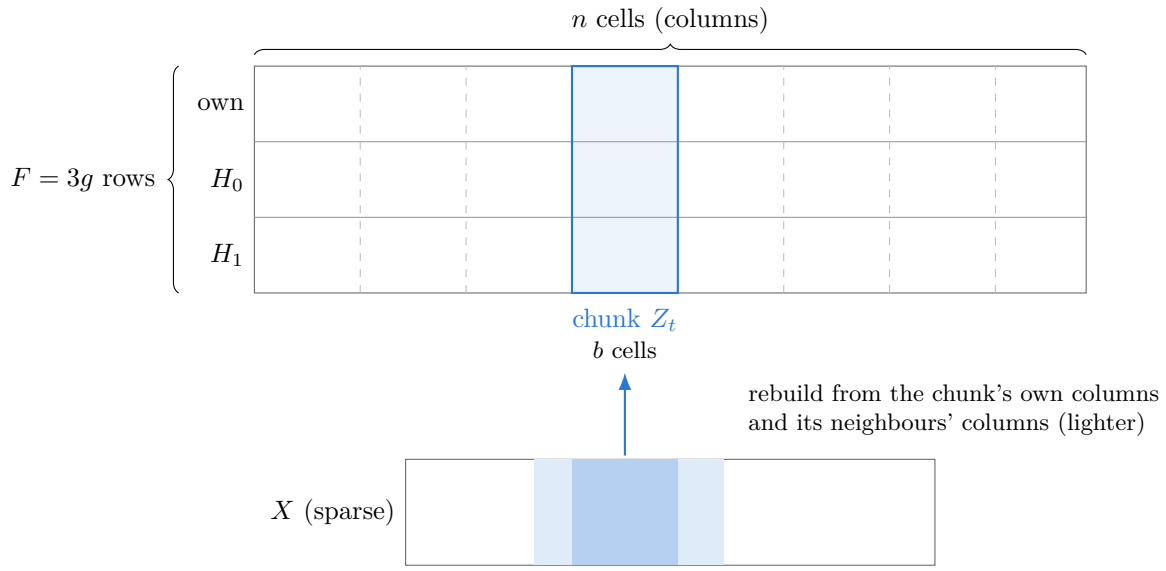


Figure 1: The BANKSY matrix Z has three blocks of g rows and one column per cell. A chunk is a vertical strip: the columns of a group of cells. It can be rebuilt from the sparse X restricted to those cells and their spatial neighbours, plus the F row means and standard deviations.

3 The key observation: any chunk of Z is cheap to rebuild

3.1 Chunks

Split the cells into groups $C_1, C_2, \ldots, C_T$ of size b (say $b = 10{,}000$), so $T = n/b$. The **chunk** $Z_t \in \mathbb{R}^{F \times b}$ is the set of columns of Z belonging to the cells in C_t: a vertical strip of the matrix (Figure 1).

To build Z_t you need, for each cell j in the chunk:

- its own column x_j of X;
- the columns x_i of its neighbours (for H_0 and H_1), with the weights w_{ij} and coefficients c_{ij};
- the global row means μ and standard deviations σ (for z-scoring). These are F numbers each, computed once (Section 5).

Nothing about cells outside the chunk and its neighbours is needed. The absolute value in the AGF is applied entry by entry, so it is no obstacle at all: compute XC for the chunk's columns, take absolute values, done. The same goes for clipping.

How much does building a chunk cost? Per cell, with 200 nonzero genes, $k = 15$ for H_0 and $k = 30$ for H_1: H_0 is about $2 \times 15 \times 200 = 6{,}000$ arithmetic operations, H_1 about $4 \times 30 \times 200 = 24{,}000$ (complex numbers cost four real multiply-adds), plus the absolute values and z-scoring of $3g = 15{,}000$ entries. Roughly 10^5 operations per cell. Keep this number; we will compare it with what we *do* with each chunk.

A chunk with $b = 10{,}000$ cells takes $F \times b \times 8 = 1.2\,\text{GB}$, which fits in memory easily.

3.2 Passes

A **pass** is one sweep through all the chunks, $t = 1, \ldots, T$: build Z_t, use it, throw it away. A pass costs one full rebuild of Z, piece by piece, but never holds more than one chunk. The

question becomes: *can we compute the PCA in a small number of passes, using each chunk once per pass?* The next subsection lists exactly what one pass can compute.

3.3 What one pass can compute

Everything below follows from one fact about matrix multiplication. The product ZA sums over the column index of Z, which is the cell index:

$$(ZA)_{fm} = \sum_{j=1}^{n} Z_{fj} A_{jm} = \sum_{t=1}^{T} \sum_{j \in C_t} Z_{fj} A_{jm}.$$

So a sum over all cells is a sum over chunks of sums over the cells in each chunk.

Fact 3.1 (Five identities)**.** Let A_t be the rows of an $n \times \ell$ matrix A that belong to chunk t, and $Q \in \mathbb{R}^{F \times \ell}$.

(i) $ZA = \sum_t Z_t A_t$. (Accumulate an $F \times \ell$ total.)

(ii) $Z^\top Q$ is the $n \times \ell$ matrix whose chunk-t rows are $Z_t^\top Q$. (Each chunk produces its own rows.)

(iii) $ZZ^\top Q = \sum_t Z_t\,(Z_t^\top Q)$. (One pass; the $n \times \ell$ intermediate is never stored.)

(iv) $Q^\top Z = [\,Q^\top Z_1,\ Q^\top Z_2,\ \ldots,\ Q^\top Z_T\,]$. (Each chunk produces its own columns.)

(v) $(Q^\top Z)(Q^\top Z)^\top = \sum_t (Q^\top Z_t)(Q^\top Z_t)^\top$. (Accumulate an $\ell \times \ell$ total.)

Proof. (i) is the display above with A in place of the generic matrix. (ii): row j of $Z^\top Q$ is $z_j^\top Q$, which involves only column j of Z. (iii): by (ii), $ZZ^\top Q = Z(Z^\top Q)$, and by (i) with $A = Z^\top Q$ this is $\sum_t Z_t (Z^\top Q)_t = \sum_t Z_t (Z_t^\top Q)$. Within a chunk you compute $P_t = Z_t^\top Q$ ($b \times \ell$), then $Z_t P_t$ ($F \times \ell$), add it to the running total and discard P_t. (iv) is (ii) transposed. (v): $(Q^\top Z)(Q^\top Z)^\top = Q^\top Z Z^\top Q = Q^\top \big(\sum_t Z_t Z_t^\top\big) Q = \sum_t (Q^\top Z_t)(Q^\top Z_t)^\top$, using $ZZ^\top = \sum_j z_j z_j^\top = \sum_t Z_t Z_t^\top$. □

Key idea

Identity (iii) is the engine of the whole method. In *one* pass, keeping only a small $F \times \ell$ running total (about 7 MB for $F = 15{,}000$, $\ell = 60$), we can multiply a block of ℓ feature-space vectors by $ZZ^\top$. We never store Z and never store the $n \times \ell$ intermediate $Z^\top Q$.

The rest of the note is about choosing *which* Q to multiply, and how few times.

4 Randomized SVD from scratch

4.1 The goal, restated

We want U_p, the top p (say 50) loading directions, out of $F = 15{,}000$ possible directions. The strategy of randomized SVD [1, 2] is:

1. Find a small set of ℓ orthonormal directions $Q \in \mathbb{R}^{F \times \ell}$ (ℓ a bit bigger than p, e.g. 60) whose span contains, as nearly as possible, the top p directions $u_1, \ldots, u_p$.

2. Squash Z onto those directions, $B = Q^\top Z \in \mathbb{R}^{\ell \times n}$, and do an ordinary SVD on that small problem.

Step 2 is easy and exact (Section 4.5). Step 1 is where the randomness and the passes go.

4.2 Why a random mixture of cells points in the important directions

Pick a random vector $\omega \in \mathbb{R}^n$, one standard normal number per cell, and compute $y = Z\omega$. That is a random weighted sum of the cells' BANKSY profiles. Using the SVD,

$$y = Z\omega = \sum_i \sigma_i \, u_i \, (v_i^\top \omega) = \sum_i \sigma_i \, \gamma_i \, u_i, \qquad \gamma_i := v_i^\top \omega.$$

Because the v_i are orthonormal and ω is standard normal, the numbers γ_i are themselves independent standard normals (a rotated Gaussian is still Gaussian; Appendix A). So y is a combination of the loading directions in which direction u_i gets coefficient $\sigma_i \gamma_i$: *random, but proportional to its singular value.* Big directions dominate.

Worked example: two directions

Suppose only two directions matter, $\sigma_1 = 10$ and $\sigma_2 = 1$. A random draw gives, say, $\gamma_1 = 0.7$, $\gamma_2 = -1.2$. Then $y = 7u_1 - 1.2u_2$: the angle between y and u_1 is $\arctan(1.2/7) \approx 10°$. One random sample already points mostly along u_1. It is not exact, because the small direction still leaks in with weight $\sigma_2 \gamma_2$.

4.3 A block of samples, and why we take more than p (oversampling)

Take ℓ random vectors at once, $\Omega \in \mathbb{R}^{n \times \ell}$, and compute $Y = Z\Omega \in \mathbb{R}^{F \times \ell}$ (identity (i): one pass). Then orthonormalise the columns of Y (QR decomposition; it is cheap because Y is small) to get Q.

If Z had exactly p nonzero singular values, any p random samples would span exactly the top-p directions (with probability one). Real Z has a long tail of small singular values, and they leak into every sample. Taking $\ell > p$ samples, **oversampling**, gives room to absorb the leakage. The extra $\ell - p$ directions soak up the strongest leaking directions ($u_{p+1}, \ldots, u_\ell$), and what still leaks in comes from $u_{\ell+1}$ onwards, which is smaller. That is why the accuracy is governed by $\sigma_{\ell+1}$ rather than σ_{p+1} (Section 7.2). A standard choice is $\ell = p + 10$.

4.4 Power iterations: making the gaps bigger

If the singular values decay slowly, as they do in single-cell data, one sample per direction is not enough. The fix is to multiply by $ZZ^\top$ a few times before orthonormalising:

$$Y = (ZZ^\top)^q \, Z\Omega = \sum_i \sigma_i^{2q+1} \, u_i \, (v_i^\top \Omega).$$

Each multiplication by $ZZ^\top$ multiplies direction u_i by σ_i^2. Ratios between singular values get raised to the power $2q + 1$: if $\sigma_{61}/\sigma_8 = 0.59$ (as in our demo), then after $q = 2$ power iterations the leak is suppressed by $0.59^5 = 0.07$, and after $q = 4$ by $0.59^9 = 0.009$. If instead $\sigma_{61}/\sigma_{50} = 0.97$, the suppression after $q = 4$ is only $0.97^9 = 0.76$: *directions whose singular values are nearly equal stay mixed.* This point decides the accuracy story in Section 7.

Why orthonormalise after every power iteration. If we just kept multiplying, every column of Y would drift towards u_1, and in floating point arithmetic the information about the smaller directions would be lost. Re-orthonormalising after each multiplication by $ZZ^\top$ keeps the ℓ columns pointing in genuinely different directions. (This is classical *subspace iteration*; the randomness is only in the starting block.) Each iteration is identity (iii), i.e. one pass: $Y \leftarrow ZZ^\top Q$, then $Q \leftarrow \mathrm{orth}(Y)$.

4.5 From Q to the PCA: the Rayleigh–Ritz step

Once Q is good, compute $B = Q^\top Z \in \mathbb{R}^{\ell \times n}$ (identity (iv)) and the small $\ell \times \ell$ matrix $BB^\top$ (identity (v)), both in one pass. Then:

1. Eigendecompose $BB^\top = \tilde{U}\,\Theta\,\tilde{U}^\top$ (an ordinary 60×60 problem). Sort the eigenvalues $\theta_1 \geq \theta_2 \geq \cdots$.
2. The estimated singular values are $\tilde{\sigma}_i = \sqrt{\theta_i}$.
3. The estimated loadings are $\tilde{U}_p = Q\,\tilde{U}_{[:,1:p]}$ $(F \times p)$.
4. The estimated embeddings are $B^\top \tilde{U}_{[:,1:p]} = Z^\top Q \tilde{U}_{[:,1:p]} = Z^\top \tilde{U}_p$, exactly formula (6) with the estimated loadings.

Why this is right: $Z \approx QQ^\top Z = QB$, and $B = \tilde{U}\Theta^{1/2}\tilde{V}^\top$ is the SVD of B, so $QB = (Q\tilde{U})\Theta^{1/2}\tilde{V}^\top$ is an SVD-shaped factorisation of the approximation of Z. This step is called **Rayleigh–Ritz**, and the $\tilde{\sigma}_i^2$ are **Ritz values**. A useful fact: Ritz values never exceed the true values, $\tilde{\sigma}_i \leq \sigma_i$, because they come from restricting $ZZ^\top$ to a subspace.

4.6 The algorithm, if Z fitted in memory

Algorithm 1 Randomized SVD with power iterations [1, Alg. 4.4 and 5.1]

Require: $Z \in \mathbb{R}^{F \times n}$, target rank p, block size $\ell \geq p$, power iterations q

Draw $\Omega \in \mathbb{R}^{n \times \ell}$ with independent standard normal entries
$Q \leftarrow \mathrm{orth}(Z\Omega)$
for $i = 1, \ldots, q$ **do**
    $Q \leftarrow \mathrm{orth}(Z(Z^\top Q))$
end for
$B \leftarrow Q^\top Z$; eigendecompose $BB^\top = \tilde{U}\Theta\tilde{U}^\top$ (sorted)
return loadings $Q\tilde{U}_{[:,1:p]}$, singular values $\sqrt{\theta_{1:p}}$, embeddings $B^\top\tilde{U}_{[:,1:p]}$

Every line that touches Z is one of the five identities, so every such line is one pass. That is the whole bridge to the chunked version.

5 The chunked algorithm, pass by pass

We now write Algorithm 1 so that Z only ever appears one chunk at a time.

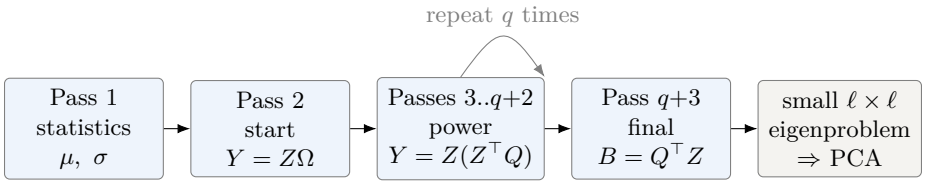


Figure 2: The passes. Every pass sweeps once through all chunks, rebuilding each one from X and the neighbour lists. Between passes only small matrices are kept ($F \times \ell$ or $\ell \times \ell$).

Pass 1: statistics. The z-scoring needs μ_f and σ_f for every row, and these depend on all cells. So the first pass builds each chunk *unscaled* (R_t, the rows of X, H_0, H_1 for the chunk's cells) and accumulates two vectors of length F:

$$S_1 = \sum_t R_t \mathbf{1}, \qquad S_2 = \sum_t (R_t \circ R_t)\mathbf{1},$$

the row sums and the row sums of squares. At the end, $\mu = S_1/n$ and $\sigma^2 = (S_2 - n\mu^2)/(n-1)$ (entry by entry). From now on every chunk is built already scaled: $Z_t = D(R_t - \mu \mathbf{1}^\top)$, and clipped if desired. For multi-sample data with `split.scale` (scaling within each sample), accumulate S_1, S_2 separately for each sample: same pass, a few more vectors.

Remark 5.1 (Numerical stability of the variance). The formula $(S_2 - n\mu^2)/(n-1)$ subtracts two large similar numbers when $\mu \gg \sigma$ and can lose digits. In double precision this is harmless for expression data (the Banksy lazy path uses the same formula). A fully robust alternative is to keep a per-chunk mean and sum of squared deviations and merge them with the pairwise formula of Chan, Golub and LeVeque [5].

Pass 2: the random start. Compute $Y = Z\Omega = \sum_t Z_t \Omega_t$ (identity (i)). The random block Ω has $n \times \ell$ entries, but it never needs to be stored: generate Ω_t for chunk t from a seed (e.g. seeded by (seed, t)), use it, discard it. It is never needed again. Then $Q \leftarrow \text{orth}(Y)$, a QR decomposition of a 15,000 $\times$ 60 matrix (a fraction of a second).

Passes 3 to q+2: power iterations. Each pass computes $Y = \sum_t Z_t(Z_t^\top Q)$ (identity (iii)), then $Q \leftarrow \text{orth}(Y)$. Per chunk:

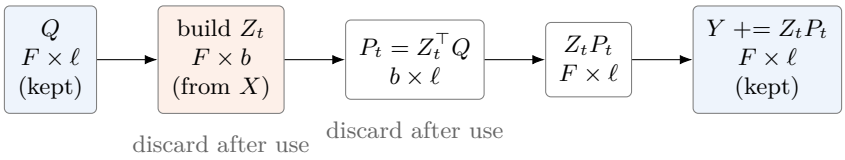


Figure 3: What happens to one chunk during a power pass. Only the blue (small) matrices survive from one chunk to the next.

Pass q+3: the final pass. For each chunk compute $B_t = Q^\top Z_t$ ($\ell \times b$), add $B_t B_t^\top$ to an $\ell \times \ell$ total G (identity (v)), and keep B_t. Its transpose, $B_t^\top$, is 60 numbers per cell, close to the final output size. After the pass, eigendecompose G and rotate: embeddings of chunk $t = B_t^\top \tilde{U}_{[:,1:p]}$. (If memory is very tight, do not keep B_t; run one more pass computing $Z_t^\top (Q\tilde{U}_{[:,1:p]})$ directly.)

Algorithm 2 Chunked randomized SVD of the BANKSY matrix

Require: sparse X, neighbour lists and weights for H_0 and H_1, λ, p, ℓ, q, chunks $C_1..C_T$

Pass 1: $S_1, S_2 \leftarrow 0$; **for** each chunk: build unscaled R_t; S_1 += $R_t\mathbf{1}$; S_2 += $(R_t \circ R_t)\mathbf{1}$

$\mu \leftarrow S_1/n$; $\sigma \leftarrow \sqrt{(S_2 - n\mu^2)/(n-1)}$; $D \leftarrow \mathrm{diag}(a_f/\sigma_f)$

Pass 2: $Y \leftarrow 0$; **for** each chunk: build Z_t; generate Ω_t from (seed, t); Y += $Z_t\Omega_t$

$Q \leftarrow \mathrm{orth}(Y)$

for $i = 1, \ldots, q$ **do** ▷ passes 3 to $q+2$

  $Y \leftarrow 0$; **for** each chunk: build Z_t; $P_t \leftarrow Z_t^\top Q$; Y += $Z_t P_t$

  $Q \leftarrow \mathrm{orth}(Y)$

end for

Final pass: $G \leftarrow 0$; **for** each chunk: build Z_t; $B_t \leftarrow Q^\top Z_t$; keep B_t; G += $B_t B_t^\top$

eigendecompose $G = \tilde{U}\Theta\tilde{U}^\top$ (sorted, decreasing)

return loadings $Q\tilde{U}_{[:,1:p]}$; singular values $\sqrt{\theta_{1:p}}$; embeddings, chunk by chunk, $B_t^\top \tilde{U}_{[:,1:p]}$

Total: $q+3$ passes. With $q = 2$ that is 5; with $q = 4$, 7. (In the earlier chat I said 6–10; that was wrong.)

5.1 Refinements

Start in feature space and save a pass. Instead of $Y = Z\Omega$ (random cell weights), start from a random orthonormal $Q_0 \in \mathbb{R}^{F\times\ell}$ and make the first pass a power pass, $Y = ZZ^\top Q_0$. For the same number of passes, the singular values get raised to the power $2q+2$ instead of $2q+1$: one free half-iteration. The demo confirms the gain (Table 4). The price is that this first pass does twice the arithmetic of the plain start pass, so per unit of arithmetic the two are the same; per pass, feature-space start is better. Passes matter when rebuilding chunks or reading X from disk (BPCells) is the bottleneck.

Merge the statistics pass into the start pass. By (5), $Z\Omega = D(R\Omega - \mu(\mathbf{1}^\top\Omega))$. In one pass you can accumulate S_1, S_2, $R\Omega$ and $\mathbf{1}^\top\Omega$, and apply μ and D afterwards. This saves a pass, but only if there is no clipping, because clipping needs μ and σ before it can be applied.

Only H_1 needs rebuilding (the hybrid). The own and H_0 blocks are exactly what the existing lazy path handles cheaply. Split $Q = [Q_{\text{own}}; Q_{H_0}; Q_{H_1}]$. In a power pass:

1. compute the own and H_0 contributions to $P = Z^\top Q$ with the lazy products $X^\top(D_1 Q_{\text{own}})$ and $W^\top X^\top(D_2 Q_{H_0})$, minus the centring terms (sparse, cheap, for all cells at once; this needs the $n \times \ell$ matrix P in memory, about 4.8 GB at 10M cells);
2. sweep the chunks once, building only the H_1 rows: add $Z_{H_1,t}^\top Q_{H_1}$ to P_t, then add $Z_{H_1,t} P_t$ to Y_{H_1};
3. compute $Y_{\text{own}} = D_1(XP - \mu_1\mathbf{1}^\top P)$ and $Y_{H_0} = D_2(X(WP) - \mu_2\mathbf{1}^\top P)$ with sparse products.

The dense arithmetic drops from $4Fn\ell$ to $4gn\ell$ per pass, three times less, at the cost of storing P. This reuses the machinery already in `R/lazy.R` and `src/banksy_lazy.cpp`, with blocks of ℓ vectors instead of one.

A free stopping rule. In each power pass we have Q and $Y = ZZ^\top Q$, hence the small matrix $Q^\top Y = Q^\top ZZ^\top Q$ for free ($F\ell^2$ operations). Its top-p eigenvalues are the Ritz values for the current subspace, and their sum is the variance the current estimate would capture.

Stop when that sum grows by less than a tolerance from one pass to the next. The demo shows the increase per pass tracks the remaining shortfall within a factor of about 3–4 (Table 5), so use a conservative tolerance. A rule based on the *vectors* rather than the variance does not work well: in the noise bulk the vectors keep rotating among themselves without the answer getting meaningfully worse or better.

Keep every block: block Krylov. Subspace iteration throws away the earlier blocks $Y_0, Y_1, \ldots$ and keeps only the latest. A block Krylov method keeps all of them and does the Rayleigh–Ritz step on their combined span, which extracts more accuracy from the same passes [7]. Here that is cheap in memory, because the blocks live in feature space: $q+1$ blocks of $F \times \ell$ are $(q+1) \times 7\,\text{MB}$. The final pass gets more expensive (it projects onto $\ell(q+1)$ directions instead of ℓ). Worth trying if the plain method needs many passes.

Parallel and distributed. Within a pass the chunks are independent. Give each thread or machine a set of chunks; each accumulates its own $F \times \ell$ partial sum; add the partial sums at the end. For bit-for-bit reproducibility, add the partial sums in a fixed order.

Several λ values at once. λ only changes the block weights a_f, i.e. the diagonal matrix D. Building the unscaled chunk R_t is the expensive part and is shared. Carry one Q per λ through the same passes and apply each D to the shared R_t.

Neighbour lists on the fly. Storing the $k = 30$ neighbour lists with complex coefficients for 10M cells takes about 6 GB. Alternatively keep only the coordinates (160 MB) and a spatial index, and recompute each chunk's neighbours as you build it. Querying a KD-tree is cheap compared with the rest of the pass.

6 Costs

Assumptions throughout: $n = 10^7$ cells, $g = 5000$ genes, $F = 15{,}000$, $p = 50$, $\ell = 60$, $q = 2$, chunk size $b = 10{,}000$, about 200 nonzero genes per cell, $k = 15$ (H_0) and $k = 30$ (H_1). A **flop** is one floating-point multiply or add. I use 100 GFLOP/s (100 billion flops per second) as a conservative effective speed for dense matrix multiplication on a multi-core CPU. The numbers are orders of magnitude, not benchmarks.

6.1 Arithmetic per pass

In a power pass, chunk t needs $Z_t^\top Q$ ($F \cdot b \cdot \ell$ multiply-adds, i.e. $2Fb\ell$ flops) and $Z_t P_t$ (another $2Fb\ell$). Over all chunks:

$$4Fn\ell = 4 \times 15{,}000 \times 10^7 \times 60 = 3.6 \times 10^{13} \text{ flops} \quad (\approx 6 \text{ minutes}).$$

The start pass and the final pass each do half of that, $2Fn\ell = 1.8 \times 10^{13}$. The statistics pass is negligible. Total for $q = 2$: $1.8 + 2 \times 3.6 + 1.8 \approx 1.1 \times 10^{14}$ flops, about 20 minutes. With the hybrid refinement, about a third of that.

6.2 Rebuild cost, and why multiplying by 60 vectors at once matters

Building a chunk costs about 10^5 flops per cell (Section 3). In a power pass, each cell then takes part in $4F\ell = 3.6 \times 10^6$ flops of useful multiplication. So rebuilding is about 3% of the

work. The rebuild is paid once per pass, and the pass serves all 60 vectors at once: its cost is **amortised** over the block.

Compare one vector at a time ($\ell = 1$, which is what `irlba` does). Useful work per cell per product is $2F = 30{,}000$ flops, but the rebuild still costs $\sim 10^5$. Rebuilding dominates, and `irlba` needs hundreds of products (476 in the demo; Section 8), each of which needs its own full rebuild of Z. Dense products with one vector are also memory-bound ("level-2 BLAS"), several times slower per flop than products with a block of vectors ("level-3 BLAS").

6.3 Memory

Held during the computation	Size at $n = 10^7$
Sparse X (the input; every method needs it, can be on disk)	∼24 GB
Neighbour lists ($k = 15$ and $k = 30$), or coordinates only	∼8 GB, or 0.16 GB
One chunk Z_t ($F \times b$)	1.2 GB
Q, Y ($F \times \ell$ each)	7 MB each
μ, σ, G	negligible
Output: $B_t^\top$ for all chunks ($n \times \ell$), then embeddings ($n \times p$)	4.8 GB, 4.0 GB

So beyond the input, the working memory is a couple of GB, and the $n \times \ell$ part is just the output, which you need anyway. (In the chat I described $n \times \ell$ as the working memory, "like irlba's V buffer"; that was imprecise.)

6.4 Comparison with the alternatives

Method	AGF?	Sweeps over data	Flops	Memory beyond X	Result
Standard: store Z, run `irlba`	yes	—	$\sim 1.5 \times 10^{14}$	1.2 TB	exact (to tolerance)
Lazy `irlba` (current)	no	none (no rebuild)	$\sim 5 \times 10^{12}$	∼5 GB	exact (to tolerance)
`irlba` + chunk rebuild	yes	∼500	$\sim 6 \times 10^{14}$, level-2	small	exact (to tolerance)
Feature covariance (Gram)	yes	2–3	$\sim 2.3 \times 10^{15}$	1.8 GB	exact
Chunked randomized SVD	yes	5 ($q = 2$)	$\sim 1.1 \times 10^{14}$	∼2 GB + output	signal PCs accurate; noise-bulk PCs approximate
Hybrid randomized SVD	yes	5 ($q = 2$)	$\sim 4 \times 10^{13}$	+ 4.8 GB for P	same as above

Table 1: Order-of-magnitude comparison at 10 million cells, 5,000 genes ($F = 15{,}000$), 50 PCs.

The feature covariance (Gram) method in a few lines. Accumulate the $F \times F$ matrix $ZZ^\top = \sum_t Z_t Z_t^\top$ in one pass ($F^2 n = 2.25 \times 10^{15}$ flops using symmetry), eigendecompose it (an $F \times F$ dense problem, $\sim F^3 = 3.4 \times 10^{12}$ flops), and project in one more pass ($Z_t^\top U_p$). Its eigenvectors are exactly the loadings U, and its eigenvalues are σ_i^2. **It is exact**: no iterations, no tolerance, and no trouble with nearly equal singular values. It costs about 60 times more arithmetic than one randomized power pass, and about 20 times more than the whole randomized run at $q = 2$. (The chat said "about 60 times more", which is the per-pass comparison.) This is what Grace and Kuai's Python pipeline does on GPUs, where $F^2 n$ is

cheap. Its cost grows with the square of the number of features: with 2,000 HVGs ($F = 6{,}000$) it is 3.6×10^{14} flops, about an hour on a CPU.

7 Accuracy

7.1 What is exact and what is not

Two different things are easy to conflate:

- **The features are exact.** Every chunk is the true BANKSY matrix, with the true AGF and true z-scoring; nothing is approximated. The demo confirms that rebuilt chunks equal the stored matrix to 7.3×10^{-13}.
- **The PCA solver is approximate.** Randomized SVD, like `irlba`, returns an approximation whose accuracy depends on q, ℓ and the spectrum of Z. `irlba` keeps iterating until a tolerance is met; randomized SVD runs a fixed number of passes.

In the chat I said "the result is the exact BANKSY PCA, not an approximation". That was wrong: only the first bullet is true.

7.2 What theory says

Halko, Martinsson and Tropp [1, Corollary 10.10] bound the error of the subspace found after q power iterations. With target rank k, oversampling $s = \ell - k \geq 2$ and a Gaussian start,

$$\mathbb{E}\,\|(I - QQ^\top)Z\| \;\leq\; \Big[\Big(1 + \sqrt{\tfrac{k}{s-1}}\Big)\sigma_{k+1}^{2q+1} + \frac{e\sqrt{\ell}}{s}\Big(\sum_{j>k}\sigma_j^{2(2q+1)}\Big)^{1/2}\Big]^{1/(2q+1)}.$$

Here $\|\cdot\|$ is the largest singular value (spectral norm). How to read it:

- The best possible error for any k-dimensional subspace is σ_{k+1} (Eckart–Young). The first term is that, times a modest constant.
- The second term is the energy of the whole tail. Without power iterations ($q = 0$) it can be much larger than σ_{k+1} when the tail is long and flat, which is the typical single-cell situation.
- Power iterations replace every σ_j by σ_j^{2q+1} inside the bracket and take the $(2q+1)$-th root outside. That makes the constants and the tail matter less and less: the bound tends to σ_{k+1} as q grows.

The rate for a particular subspace. The classical analysis of subspace iteration ([3, 4]; for the randomized version see [2]) gives a simpler, more intuitive rule. The angle between the true top-k directions and the span of Q shrinks roughly like

$$\Big(\frac{\sigma_{\ell+1}}{\sigma_k}\Big)^{2q+1}. \tag{7}$$

This explains everything we will see:

- **Oversampling** enters through $\sigma_{\ell+1}$: a larger ℓ compares σ_k with a smaller singular value, so the ratio is smaller and convergence is faster.

- **Strong, separated components** (signal PCs, far above the noise) have $\sigma_{\ell+1}/\sigma_k$ well below 1 and converge in a few passes.
- **Components in a flat noise bulk** have $\sigma_{\ell+1}/\sigma_k$ close to 1 and barely move.

Figure 7 checks (7) on the demo data.

7.3 Signal PCs and the noise bulk

The spectrum of a BANKSY matrix (Figure 5) has a few large singular values from real structure, followed by a long, nearly flat bulk of noise components. With $n \gg F$ the bulk is very narrow: random-matrix theory puts the noise singular values within a factor of about $(1 \pm \sqrt{F/n})$ of each other, which is $\pm 4\%$ at $n = 10^7$, $F = 15{,}000$. Most of PCs 1–50 in a typical analysis can sit inside that bulk.

Inside the bulk, the individual directions are not determined by the biology. Which 38 of the thousands of nearly equivalent noise directions make it into "PCs 13–50" depends on tiny differences in singular values. A careful Lanczos solver (`irlba` to tolerance) will find the same ones every time, because they *are* defined mathematically. A few passes of randomized iteration will not: it returns other noise directions that capture almost the same variance. That is the honest description of the method: *leading PCs accurate, trailing bulk PCs replaced by equally good but different noise directions.*

7.4 What matters downstream

Clustering and UMAP use the distances between cells' embeddings. By (6), the distance between cells i and j is

$$\|U_p^\top z_i - U_p^\top z_j\| = \|U_p U_p^\top (z_i - z_j)\|,$$

which depends only on the *subspace* spanned by the loadings ($U_p U_p^\top$ is the projector onto it), not on the individual PCs. So a solver that rotates PCs among themselves inside the top-p subspace changes nothing downstream. What can change things is the subspace itself: if some bulk directions are swapped for other bulk directions, distances change by the noise in those directions. Section 8 measures how much: about as much as using 35–40 instead of 50 exact PCs.

7.5 Floating point

Computing $ZZ^\top Q$ in one go, without orthonormalising between $Z^\top$ and Z, loses directions whose singular values are below about $\sqrt{\varepsilon}\,\sigma_1 \approx 10^{-8}\sigma_1$, where $\varepsilon \approx 10^{-16}$ is double-precision rounding [1, Sec. 4.5]. PCs 1–50 have $\sigma_{50}/\sigma_1 \approx 0.2$ in the demo, nowhere near that, so fusing the two products is safe. Similarly, eigendecomposing $BB^\top$ (rather than taking an SVD of B) squares the singular values, which is harmless for the top 50.

8 A numerical demonstration

Everything in this section comes from `demo/agf_rsvd_demo.py` (Python; runs in about three minutes). All numbers in the text are pulled from the script's output, so they match the figures.

8.1 The dataset

10,000 cells placed uniformly in a square tissue, 400 genes, 87% zero counts. Five spatial domains (four wavy layers and a round inclusion, Figure 4a), ten cell types with domain-specific mixtures, eight domain gene sets, eight smooth spatial "programs", ten gradient genes and 150 noise genes. Counts are Poisson, library-size normalised and log-transformed. BANKSY features are built exactly as in Section 1: $k = 15$ for H_0, $k = 30$ for the centred AGF, `kNN_median` weights, $\lambda = 0.8$, giving $F = 1{,}200$ features. The full matrix is small enough to store, so we can compute the exact PCA with a dense SVD and compare.

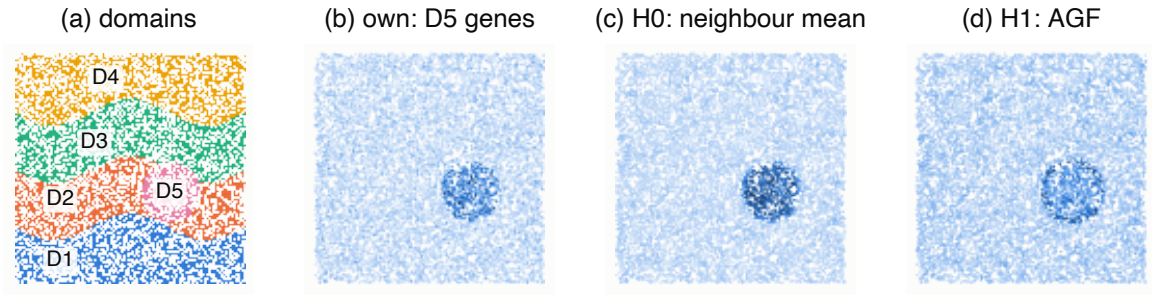


Figure 4: The simulated tissue. (a) The five labelled domains. (b)–(d) Summed over the eight genes specific to the round inclusion D5: own expression, the neighbour mean H_0, and the AGF H_1. H_0 is a smoothed copy of (b). H_1 is not: it responds to how uneven expression is around each cell, which is largest in and around the inclusion, where expression is high and so is its cell-to-cell noise.

8.2 Checks of the construction

- The compact AGF form (3) equals the textbook form (1) to 1.7×10^{-16}. The coefficients of every cell sum to zero (largest sum 2.0×10^{-16}).
- A chunk rebuilt from X and the neighbour lists equals the corresponding columns of the stored Z to 7.3×10^{-13} (the small difference comes from computing the variance by the sum-of-squares formula).
- The chunked algorithm (20 chunks of 500 cells, $q = 2$, 5 passes) and the in-memory algorithm with the same random numbers agree: singular values to 9.4×10^{-15}, loadings subspace to 3.2×10^{-13}. **Chunking changes nothing.**
- The exact alternative (feature covariance, computed chunk by chunk) matches the dense SVD: singular values to 5.3×10^{-15}, top-50 subspace to 7.2×10^{-13}, kNN graph overlap 1.0000.

8.3 The spectrum

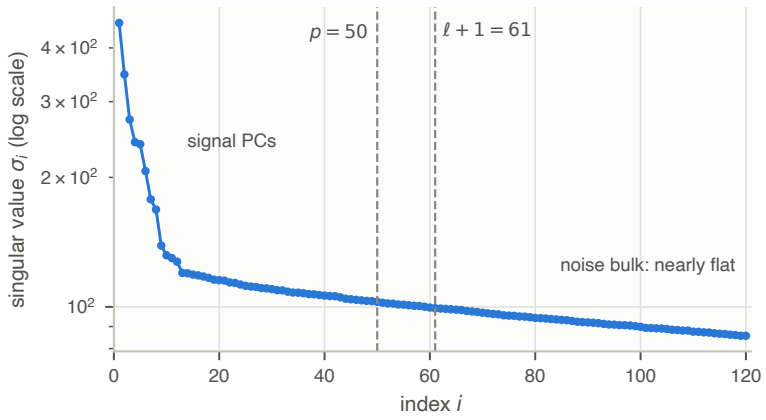


Figure 5: Singular values of the demo BANKSY matrix. About a dozen signal components, then a long, nearly flat noise bulk.

There is a clear gap after PC 8 ($\sigma_8 = 168.4$, $\sigma_9 = 138.8$, ratio 0.824), a few more structured components, and then a flat bulk: $\sigma_{51}/\sigma_{50} = 0.995$, $\sigma_{21}/\sigma_{20} = 0.998$. The rate ratios from (7) with $\ell = 60$ are $\sigma_{61}/\sigma_8 = 0.590$ (fast) and $\sigma_{61}/\sigma_{50} = 0.966$ (very slow).

8.4 Convergence

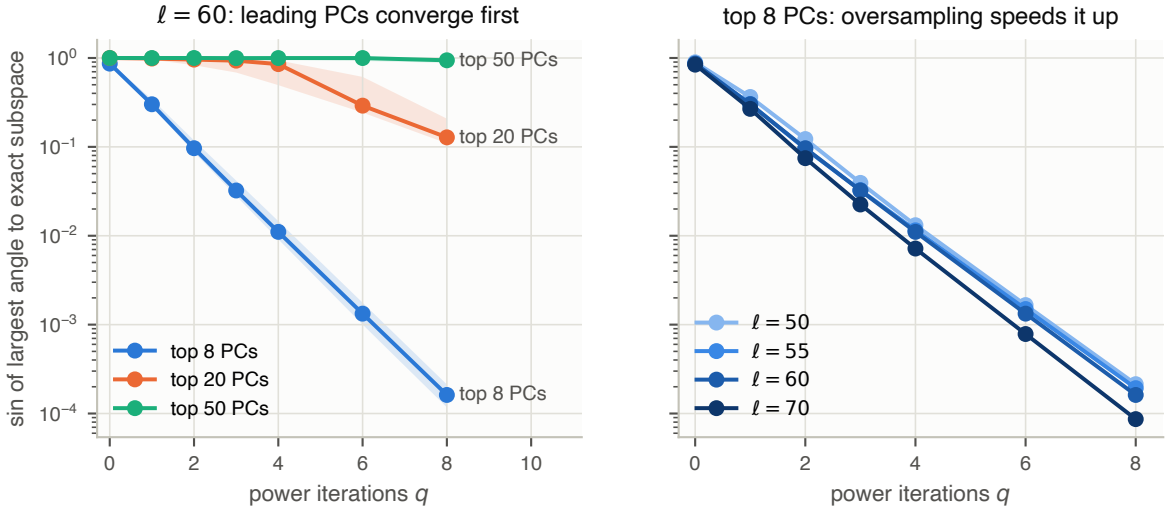


Figure 6: Sine of the largest angle between the estimated and the exact top-k loading subspace (0 = identical, 1 = some direction completely missed). Median over five random seeds; the shaded band is the range. Left: $\ell = 60$, for the top 8, 20 and 50 PCs. Right: top 8 PCs for four block sizes.

q	passes	$\sin\theta_8$	$\sin\theta_{20}$	$\sin\theta_{50}$	err. $\sigma_{1..8}$	err. $\sigma_{1..50}$	var. shortfall	kNN Jaccard
0	3	0.864	1.000	1.000	0.344	0.370	0.433	0.51
1	4	0.302	0.985	0.999	0.0289	0.157	0.0979	0.64
2	5	0.0966	0.959	1.000	2.9×10^{-3}	0.0888	0.0452	0.70
3	6	0.0323	0.933	0.997	3.3×10^{-4}	0.0590	0.0255	0.73
4	7	0.0111	0.853	0.998	3.9×10^{-5}	0.0424	0.0156	0.76
6	9	1.3×10^{-3}	0.291	0.999	5.7×10^{-7}	0.0255	7.0×10^{-3}	0.80
8	11	1.6×10^{-4}	0.128	0.942	8.5×10^{-9}	0.0175	3.7×10^{-3}	0.83

Table 2: $\ell = 60$, median over five seeds. $\sin\theta_k$: largest-angle sine for the top-k subspace. err.: largest relative error among those singular values. Variance shortfall: fraction of the top-50 variance $\sum_{i\le 50}\sigma_i^2$ not captured. kNN Jaccard: mean overlap of each cell's 20 nearest neighbours in the 50-PC embedding, estimated against exact.

Reading Figure 6 and Table 2:

- **The top 8 PCs converge fast**: $\sin\theta_8 = 0.0966$ at $q = 2$, 0.0111 at $q = 4$, 1.6×10^{-4} at $q = 8$. Their singular values are right to 3×10^{-3} at $q = 2$ and 4×10^{-5} at $q = 4$.
- **The top 50 do not converge**: $\sin\theta_{50}$ stays near 1 even at $q = 8$. Some noise direction from the exact top 50 is always missing, replaced by a different noise direction.
- **The variance is nearly all there**: the estimated top-50 subspace captures all but 0.0452 of the variance at $q = 2$ and 0.0156 at $q = 4$ (Figure 8). The replacement noise directions are almost as large as the ones they replace.
- **Oversampling helps**, as (7) predicts (Figure 6, right; Figure 8).

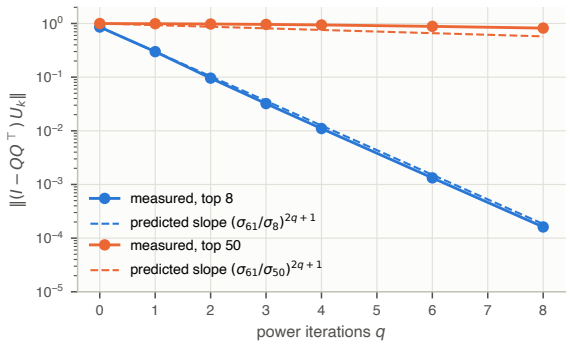


Figure 7: Measured capture error $\|(I - QQ^\top)U_k\|$ against the predicted rate $(\sigma_{61}/\sigma_k)^{2q+1}$ (scaled to match at $q = 0$; the point is the slope). $\ell = 60$.

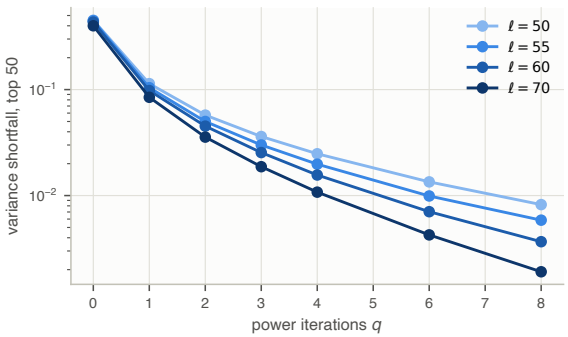


Figure 8: Fraction of the top-50 variance not captured, for four block sizes. Median over five seeds.

Figure 7 shows the rate rule (7) working: for the top 8 the measured error falls with exactly the predicted slope, and for the top 50 it barely falls, also as predicted.

8.5 Does it change the kNN graph and the clusters?

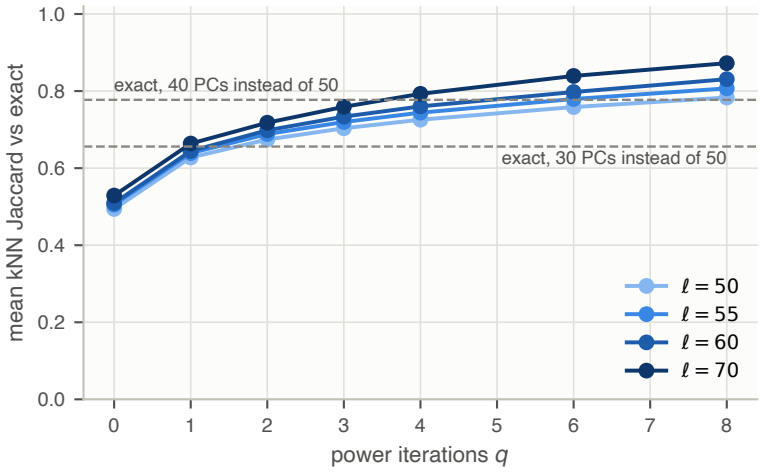


Figure 9: Mean overlap (Jaccard) between each cell's 20 nearest neighbours in the estimated 50-PC embedding and in the exact one. Dashed: the same overlap when the exact embedding uses 40 or 30 PCs instead of 50.

The kNN overlap with the exact embedding is 0.70 at $q = 2$ and 0.76 at $q = 4$ (Figure 9). On its own that number looks alarming. For scale: using the *exact* PCA but keeping 40 PCs instead of 50 gives 0.78, and 30 PCs gives 0.66. The choice of 50 PCs is itself arbitrary, and the kNN graph is sensitive to it, because within a group of similar cells the nearest neighbours are decided by noise dimensions. The randomized method perturbs the kNN graph about as much as choosing 35–40 instead of 50 PCs.

q	passes	ARI vs exact (median)	ARI vs exact (worst of 5)
0	3	0.71	0.64
1	4	0.76	0.74
2	5	0.81	0.78
4	7	0.84	0.80

Table 3: Agreement (adjusted Rand index) between clusters from the estimated and the exact embedding. Seurat-style clustering: shared-nearest-neighbour weights on the 20-NN graph, Louvain, resolution 0.1 (10 clusters). For reference: the exact embedding clustered with different Louvain seeds agrees with itself at ARI 0.82–0.98, and exact with 30 instead of 50 PCs gives 0.74.

Clusters (Table 3) agree with the exact clustering at ARI 0.81 ($q = 2$) to 0.84 ($q = 4$). That is just below the range of Louvain's own seed-to-seed variation (0.82–0.98) and better than the difference between 30 and 50 exact PCs (0.74). A small but measurable effect, which shrinks with more passes.

8.6 The refinements, checked

	$\sin\theta_8$		variance shortfall	
passes	cell start	feature start	cell start	feature start
4	0.302	0.143	0.0979	0.0636
5	0.0966	0.0432	0.0452	0.0333
6	0.0323	0.0141	0.0255	0.0200
7	0.0111	4.8×10^{-3}	0.0156	0.0129
8	3.8×10^{-3}	1.6×10^{-3}	0.0102	8.7×10^{-3}

Table 4: Same number of passes (including the statistics pass), two ways to start (Section 5.1). Starting in feature space wins at every pass count. $\ell = 60$, median over five seeds.

power pass	monitor (increase in captured variance)	true remaining shortfall
2	0.391	0.101
3	0.0577	0.0457
4	0.0206	0.0256
5	9.9×10^{-3}	0.0159
6	5.4×10^{-3}	0.0105
7	3.1×10^{-3}	7.4×10^{-3}
8	2.0×10^{-3}	5.5×10^{-3}
9	1.3×10^{-3}	4.2×10^{-3}

Table 5: The free stopping monitor against the truth ($\ell = 60$, one seed). The monitor is always within a factor of about 3–4 of the remaining shortfall, and underestimates it from pass 4 on, so use a tolerance three to four times smaller than the shortfall you can accept.

8.7 How many products does a Lanczos solver need?

On the same matrix, ARPACK (a Lanczos-type solver, called through `scipy`) needed 213 forward and 263 transpose products (476 in total) to find 50 singular triplets at tolerance 10^{-5}, and found them to machine precision. With the AGF and no algebraic shortcut, each of those products would be a full rebuild of Z. That is the number behind "hundreds of passes".

8.8 Caveats

This is a small simulation: $F = 1{,}200$, not 15,000, and $n = 10{,}000$, not 10^7. It says nothing about runtime. Real data have more structured components and, with much larger n, an even narrower noise bulk, so the qualitative picture (signal PCs fast, bulk PCs not resolved) should hold, but the numbers will differ. The right test is on a real dataset that still fits in memory (Section 9).

9 What this means for Banksy

1. **AGF without storing the BANKSY matrix is feasible.** Chunk recomputation is exact and simple. It needs no new mathematics, only the observation that any group of cells' columns can be rebuilt from X and the neighbour lists.

2. **Which solver depends on what "equivalent" must mean.**

- If the requirement is "same biology" (same leading PCs, same clusters up to clustering noise): chunked randomized SVD, ideally the hybrid version (lazy products for own and H_0, rebuilt chunks only for H_1). $\ell = p+10$ to $p+20$, start in feature space, stop with the captured-variance monitor (in the demo 4–6 power passes). Cost is in the same range as the current lazy path.
- If the requirement is "numerically the same PCA as the standard path", for example to pass the same kNN-Jaccard equivalence check as the lazy path (0.9999): the feature-covariance method gives it exactly in 2–3 passes. Fine with HVGs ($F = 6{,}000$: about an hour on CPU), heavy for the full 5K panel on CPU, cheap on GPU.

3. **How to validate.** On a real dataset that fits in memory (standard path available), compare: angles for the leading, well-separated PCs (should be tiny); captured variance (should be within a percent or two); kNN Jaccard, *against the reference of using fewer exact PCs*; and clustering ARI, *against the clustering's own seed-to-seed variation.* Without those two references the kNN and ARI numbers are impossible to interpret.

4. **Is it worth it?** Joseph's experience (issue #63) is that results are fairly robust with or without the AGF. A quick test, before building anything, is to compare $M = 0$ and $M = 1$ domain segmentation on a dataset where the standard path still fits, and see whether the AGF visibly changes the domains.

Side finding. As noted in Section 1, the standard path computes $H_0 = |XW|$ while the lazy path uses XW. They agree for nonnegative data and disagree for signed data (z-scored protein, e.g. CODEX). Worth checking with Joseph which is intended.

A Linear algebra refresher

Orthonormal. Vectors $q_1, \ldots, q_\ell$ are orthonormal if each has length 1 and every pair is perpendicular: $q_i^\top q_j = 1$ if $i = j$, else 0. For a matrix Q with orthonormal columns, $Q^\top Q = I$.

QR decomposition and "orth". Any $F \times \ell$ matrix Y can be written $Y = QR$ with Q orthonormal ($F \times \ell$) and R upper triangular ($\ell \times \ell$). The columns of Q span the same space as the columns of Y. orth(Y) means "this Q". It costs about $2F\ell^2$ flops, negligible here.

Projector. $QQ^\top$ maps any vector to its closest point in the span of Q. $(I - QQ^\top)x$ is what is left over: the part of x outside the span.

SVD. $Z = U\Sigma V^\top$ with U and V orthonormal and Σ diagonal with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. $Zv_i = \sigma_i u_i$ and $Z^\top u_i = \sigma_i v_i$. $ZZ^\top = U\Sigma^2 U^\top$, so the eigenvectors of $ZZ^\top$ are the u_i and its eigenvalues are σ_i^2. Eckart–Young: the best approximation of Z by a matrix of rank k is $\sum_{i \leq k} \sigma_i u_i v_i^\top$, with spectral-norm error σ_{k+1}.

Principal angles. For two k-dimensional subspaces with orthonormal bases U and $\tilde{U}$, the largest principal angle θ satisfies $\sin\theta = \|(I - \tilde{U}\tilde{U}^\top)U\|$. $\sin\theta = 0$: identical subspaces. $\sin\theta = 1$: some direction of one is perpendicular to all of the other.

Rotated Gaussians are Gaussian. If ω has independent standard normal entries and V is orthonormal, then $V^\top\omega$ also has independent standard normal entries. (The standard normal distribution is the same in every direction.) This is why $\gamma_i = v_i^\top\omega$ in Section 4 are independent standard normals.

Ritz values are lower bounds. For orthonormal Q, the eigenvalues of $Q^\top ZZ^\top Q$ are at most the corresponding eigenvalues of $ZZ^\top$ (Cauchy interlacing). So estimated singular values never exceed the true ones, and the captured variance can only approach the best value from below.

B Glossary

AGF Azimuthal Gabor filter: a feature measuring how lopsided a gene's expression is around a cell, Eq. (1).

BLAS levels Matrix–vector products (level 2) are limited by memory speed; matrix–matrix products (level 3) reuse data in cache and run several times faster per flop.

Block A set of ℓ vectors processed together, here as the columns of an $F \times \ell$ or $n \times \ell$ matrix.

Chunk The columns of Z for a group of b cells.

Chunk recomputation Building a chunk from X and the neighbour lists every time it is needed, instead of storing it.

Embedding Each cell's coordinates on the top p PCs, $Z^\top U_p$.

Flop One floating-point multiply or add.

Gram / feature covariance The $F \times F$ matrix $ZZ^\top$; its eigenvectors are the loadings.

Lanczos / irlba Iterative SVD solvers that use one forward and one transpose product per step, hundreds of steps in total.

Loadings The directions $u_i \in \mathbb{R}^F$ of the PCs in feature space.

Noise bulk The long, nearly flat tail of singular values that comes from noise rather than structure.

Oversampling Using $\ell > p$ random samples so that leakage from the tail is absorbed by the extra directions.

Pass One sweep through all chunks.

Power iteration Multiplying the current block by $ZZ^\top$ (and re-orthonormalising) to suppress small directions.

Rayleigh–Ritz Projecting the problem onto the span of Q and solving the small problem exactly.

Spectral norm The largest singular value of a matrix, i.e. the most it can stretch a unit vector.

Subspace iteration Repeated multiplication by $ZZ^\top$ with re-orthonormalisation; randomized SVD is subspace iteration from a random start.

z-scoring Subtracting each row's mean and dividing by its standard deviation.

C The core of the chunked algorithm in code

From `demo/agf_rsvd_demo.py` (Python). `builder.scaled(idx)` rebuilds the chunk for cells `idx` from X and the neighbour lists.

```
def rsvd_chunked(builder, chunks, p, ell, q, seed):
    stats_pass(builder, chunks)                       # pass 1: mu, sigma
    Y = np.zeros((F, ell))                            # pass 2: Y = Z Omega
    for t, idx in enumerate(chunks):
        Zt = builder.scaled(idx)
        Y += Zt @ omega_chunk(seed, t, len(idx), ell)
    Q, _ = np.linalg.qr(Y)
    for _ in range(q):                                # power passes
        Y = np.zeros((F, ell))
        for idx in chunks:
            Zt = builder.scaled(idx)
            Y += Zt @ (Zt.T @ Q)                      # identity (iii)
        Q, _ = np.linalg.qr(Y)
    Bt, G = [], np.zeros((ell, ell))                  # final pass
    for idx in chunks:
        Bc = Q.T @ builder.scaled(idx)
        Bt.append(Bc); G += Bc @ Bc.T                 # identity (v)
    evals, Ut = np.linalg.eigh(G)
    order = np.argsort(evals)[::-1]; Ut = Ut[:, order]
    loadings = Q @ Ut[:, :p]
    embeddings = np.vstack([Bc.T @ Ut[:, :p] for Bc in Bt])
    return loadings, np.sqrt(evals[order][:p]), embeddings
```

References

[1] N. Halko, P.-G. Martinsson, J. A. Tropp. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* 53(2):217–288, 2011.

[2] P.-G. Martinsson, J. A. Tropp. Randomized numerical linear algebra: foundations and algorithms. *Acta Numerica* 29:403–572, 2020.

[3] G. H. Golub, C. F. Van Loan. *Matrix Computations*, 4th ed. Johns Hopkins University Press, 2013 (orthogonal / subspace iteration).

[4] Y. Saad. *Numerical Methods for Large Eigenvalue Problems*, revised ed. SIAM, 2011 (subspace iteration).

[5] T. F. Chan, G. H. Golub, R. J. LeVeque. Algorithms for computing the sample variance: analysis and recommendations. *The American Statistician* 37(3):242–247, 1983.

[6] J. Baglama, L. Reichel. Augmented implicitly restarted Lanczos bidiagonalization methods. *SIAM J. Sci. Comput.* 27(1):19–42, 2005 (the algorithm behind `irlba`).

[7] C. Musco, C. Musco. Randomized block Krylov methods for stronger and faster approximate singular value decomposition. *NeurIPS*, 2015.

[8] V. Singhal et al. BANKSY unifies cell typing and tissue domain segmentation for scalable spatial omics data analysis. *Nature Genetics* 56:431–441, 2024.